

Tree Aggregation with plyxp and tree_vctr

Justin Landis

```
library(TreeSummarizedExperiment)
library(plyxp)
library(treeClustDemo)
```

Data Setup

```
set.seed(42)
se <- SummarizedExperiment(
  assays = list(counts = matrix(rpois(10000, 10), 1000, 10)),
  rowData = DataFrame(
    feature = sprintf("f%04i", 1:1000),
    length = sample(1e3:1e5, 1000, replace = TRUE)
  ),
  colData = DataFrame(
    sample = sprintf("s%02i", 1:10)
  )
)

.rowtree <- ape::rtree(1000)
tse <- TreeSummarizedExperiment(
  assays = assays(se),
  rowData = rowData(se),
  colData = colData(se),
  rowTree = .rowtree,
  rowNodeLab = sample(.rowtree$tip.label)
)
```

The Goal

Provide some tidyomics styled abstraction over TreeSummarizedExperiment (treeSE). As noted from a prior meeting, the main utility for treeSE is the ability to not only store and map arbitrary trees to the rows or columns of the SummarizedExperiment (SE), but also provide utility functions for summarizing data to some level of the tree nodes.

In tidyomics, there are two stylings to approach this abstraction. We could apply the relaxed styling of TidySummarizedExperiment (tidySE) or we could approach from a more verbose styling of plyxp. Given my bias, I will be demonstrating the approach from plyxp

Background treeSE

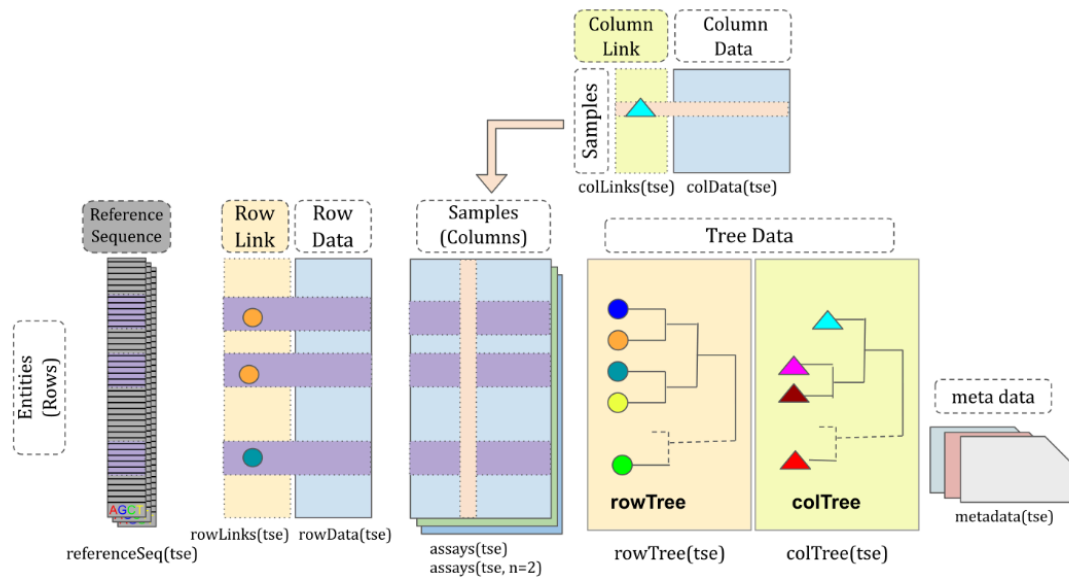


Figure 1: The structure of a treeSE object as shown from the Bioconductor vignette.

In the above figure, we can see how the treeSE package chose to map arbitrary phylo trees to data within an SE object. The data is mapped by either the “rowLink” or “colLink” data structure.

```
rowTree(tse, whichTree = 1L)
```

Phylogenetic tree with 1000 tips and 999 internal nodes.

Tip labels:

t711, t52, t122, t553, t403, t32, ...

Rooted; includes branch length(s).

```
rowLinks(tse)
```

LinkDataFrame with 1000 rows and 5 columns

	nodeLab	nodeLab_alias	nodeNum	isLeaf	whichTree
	<character>	<character>	<integer>	<logical>	<character>
1	t868	alias_693	693	TRUE	phylo
2	t463	alias_397	397	TRUE	phylo
3	t521	alias_275	275	TRUE	phylo
4	t900	alias_714	714	TRUE	phylo
5	t211	alias_710	710	TRUE	phylo

...	...	...	...	...	...
996	t470	alias_384	384	TRUE	phylo
997	t303	alias_250	250	TRUE	phylo
998	t264	alias_370	370	TRUE	phylo
999	t409	alias_186	186	TRUE	phylo
1000	t472	alias_41	41	TRUE	phylo

The key to this structure is the “LinkDataFrame” class, which is meant to be parallel to the associated rowData or colData of the SE object. In other words, the *i*th observation of the rowLinks will hold a mapping to some node of the rowTree, which in turns maps to the *i*th row of the SE object.

Applying plyxp to tree Structures

The work at present is not a complete implementation of tidyomics for the treeSE class, however it does provide building blocks that could lead to its implementation.

The reason a complete implementation hasn’t been made is that the internals plyxp relies on to do it properly have not yet been exported and are not yet stable.

That being said, plyxp works as intended with normal vectors or S4Vectors within the rowData or colData. Thus this proof of concept abstracts the mapping of the LinkDataFrame into a similar custom structure, a tree_vctr, in order to demonstrate the functionality.

Anatomy of a tree_vctr

A tree_vctr is an S3 vector built on top of the “vctrs_rcrd” class from the vctrs package. It is meant to be a light weight, self contained structure, that provides similar functionality of the rowLinks() and rowTree() functions.

```
tree_vec <- tree_vctr(.rowtree)
# attribute: tree -> list of hclust objects
# data: data.frame(which_tree: <factor>, node: <integer>)
#   which_tree is factor incoding where
#   - level 1 maps to tree index 1,
#     level 2 maps to tree index 2, ... etc
#   - levels are computed by `digest::digest`
#     the hclust objects
#
#   node is an integer representing the node of tree i
#   - This number depends on the size of the tree.
#     If tree 1 has N leafs, then node cannot be
#     greater than M = N * (N - 1)
#
#   The output for each element is formatted as:
#     <which_tree>:<node_id>
c(head(tree_vec), tail(tree_vec))
```

```
<tree_vctr[12]>
 [1] 1:1    1:2    1:3    1:4    1:5    1:6    1:995  1:996  1:997  1:998
[11] 1:999  1:1000
# encoding by order
# n tree(s): 1
```

Because it is built on top of the `vctrs` package, standard book keeping is retained as you subset, or reorder the vector.

The intent of this object is to sit alongside other vectors to annotate these objects. This differs from `treeSE` in that the `LinkDataFrame` and the `rowTree()` list are separate objects that live on a `treeSE` object. By themselves, they are not informative. The `tree_vctr` abstracts this metadata as a stand-alone vector that can be used by composition with any standard SE object.

Warning

At the moment, `tree_vctr` only works with `hclust` objects as our use case involves this type of clustering. This also differs from `treeSE` in that the expected tree structure must be a phylo object. We may be able to make this vector class agnostic towards the trees it holds.

The `tree_vec` object is not yet properly mapped to our data. We can use data from our `tse` object to properly map the nodes.

```
rowData(se)$tree <- tree_vec[rowLinks(tse)$nodeNum]
rowData(se)
```

```
DataFrame with 1000 rows and 3 columns
  feature    length    tree
  <character> <integer> <tree_vctr>
1      f0001     78114 t868-1
2      f0002     25742 t463-1
3      f0003     28772 t521-1
4      f0004      2996 t900-1
5      f0005     57250 t211-1
...      ...      ...      ...
996     f0996     28440 t470-1
997     f0997     26166 t303-1
998     f0998     13306 t264-1
999     f0999     67895 t409-1
1000    f1000     41477 t472-1
```

Utility functions for `tree_vctr`

```
.tvec <- sample(tree_vec, 10)
tree_id(.tvec) # get tree index as integer
```

```
[1] 1 1 1 1 1 1 1 1 1 1
```

```
node_id(.tvec) # get node id
```

```
[1] 397 262 874 829 227 473 741 186 295 981
```

```
tree(.tvec) # expects only 1 tree
```

```
Call:
phylo_to_hclust(x = x)

Cluster method      : unknown
Number of objects: 1000
```

```
trees(.tvec) # returns all trees
```

```
[[1]]

Call:
phylo_to_hclust(x = x)

Cluster method      : unknown
Number of objects: 1000
```

```
is_inner(.tvec) #logical vector if node is inner node
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
tail(inner(.tvec)) # for each tree, generate all inner nodes
```

```
<tree_vctr[6]>
[1] 1:1994 1:1995 1:1996 1:1997 1:1998 1:1999
# encoding by order
# n tree(s): 1
```

```
.parent <- parent_nodes(.tvec) # get the parent of a node
.parent
```

```
<tree_vctr[10]>  
[1] 1:1155 1:1013 1:1423 1:1023 1:1101 1:1492 1:1844 1:1098 1:1448 1:1467  
# encoding by order  
# n tree(s): 1
```

```
child_nodes(.parent) # get both children from a parent
```

```
<tree_vctr[20]>  
[1] 1:396 1:397 1:261 1:262 1:1291 1:874 1:828 1:829 1:227 1:228  
[11] 1:1459 1:473 1:741 1:742 1:185 1:186 1:295 1:296 1:981 1:1460  
# encoding by order  
# n tree(s): 1
```

end